## رشته‌ها (Strings) در زبان برنامه‌نویسی پایتون

رشته (String) یکی از پرکاربردترین انواع داده (Data Type) در زبان پایتون است. رشته برای نگهداری و پردازش داده‌های متنی مانند نام افراد، آدرس‌ها، شماره تلفن، پیام‌ها، ایمیل‌ها و ... استفاده می‌شود.

در پایتون، رشته مجموعه‌ای از کاراکترها (Characters) است که داخل علامت نقل قول قرار می‌گیرد.

یک رشته می‌تواند شامل:

- حروف
- اعداد
- فاصله (Space)
- علائم نگارشی
- نمادها

باشد.

### ۱- ایجاد یک رشته(Creating Strings)

در پایتون، رشته‌ها را می‌توان با استفاده از نقل قول‌های تکی (') یا دوتایی (") ایجاد کرد.

**استفاده از کوتیشن دوتایی**

```
name = "Python Programming"
print(name)
```

**استفاده از کوتیشن تکی**

```
language = 'Python'
print(language)
```

هر دو روش کاملاً صحیح هستند

### ۲- استفاده از نقل قول داخل رشته

اگر داخل رشته از علامت ' استفاده می‌کنید، بهتر است رشته را با " "بنویسید

```
text = 'He said "Hello"'
print(text)
```

### ۳- رشته‌های چندخطی(Multiline Strings)

گاهی لازم است یک متن در چند خط نوشته شود در این حالت از **سه کوتیشن دوتایی** (""" """)یا **سه کوتیشن تکی** (''' ''')استفاده می‌کنیم.

**استفاده از سه کوتیشن دوتایی(Triple Double Quotes)**

```
text = """
Python is an easy language.
It is powerful.
It is popular.
"""
print(text)
```

خروجی

```
Python is an easy language.
It is powerful.
It is popular.
```

**استفاده از سه کوتیشن تکی**

```
text = '''
Python
Java
C++
'''
print(text)
```

**مزایای Multiline Strings**

- نوشتن متن‌های طولانی
- ذخیره توضیحات
- تولید گزارش‌ها
- ایجاد قالب‌های متنی (Templates)

**٤- رشته‌ها آرایه هستند(Strings are Arrays)**

در پایتون، رشته‌ها مانند یک آرایه از کاراکترها رفتار می‌کنند هر کاراکتر دارای یک شماره (Index) است.

مثال

Python

**P y t h o n**

0 1 2 3 4 5

**دسترسی به یک کاراکتر**

```
language = "Python"
print(language[0])
```

خروجی

P

```
print(language[3])
```

خروجی

h

**اندیس منفی(Negative Index)**

می‌توان از انتهای رشته نیز به کاراکترها دسترسی داشت

```
language = "Python"
print(language[-1])
```

خروجی

```
n
```

```
print(language[-2])
```

خروجی

```
o
```

### ۵- پیمایش رشته(Looping Through a String)

از آنجا که رشته یک آرایه از کاراکترها است، می‌توان روی آن پیمایش انجام داد.

```
for letter in "Python":
   print(letter)
```

خروجی

```
P
y
t
h
o
n
```

### ٦- طول رشته(String Length)

برای محاسبه تعداد کاراکترهای یک رشته از تابع len()استفاده می‌شود

**مثال**

```
language = "Python"
print(len(language))
```

خروجی

6

مثال دیگر

```
name = "Ali Ahmadi"
print(len(name))
```

خروجی

10

**نکته:** فاصله (Space) نیز یک کاراکتر محسوب می‌شود.

### ۷- بررسی وجود یک عبارت در رشته(Check String)

برای بررسی اینکه آیا یک کلمه یا عبارت در رشته وجود دارد یا خیر، از عملگر in استفاده می‌شود

**مثال**

```
text = "Python is easy"
print("Python" in text)
```

خروجی

True

مثال دیگر

```
text = "Python is easy"
print("Java" in text)
```

خروجی

False

**۸- استفاده از in در شرط‌ها**

```
text = "Python Programming"
if "Python" in text:
   print("Word Found")
```

**۹- بررسی عدم وجود یک عبارت(Check if NOT)**

برای بررسی اینکه یک عبارت در رشته وجود ندارد از عملگر not in استفاده می‌شود

**مثال**

```
text = "Python Programming"
print("Java" not in text)
```

خروجی

True

مثال دیگر

```
text = "Python Programming"
print("Python" not in text)
```

خروجی

False

**۱۰-استفاده از not in در شرط‌ها**

```
text = "Python Programming"
if "Java" not in text:
   print("Word Not Found")
```

**مثال جامع**

```
language = "Python Programming"
print(language)
print(language[0])
print(language[-1])
print(len(language))
print("Python" in language)
print("Java" in language)
print("Java" not in language)
```

خروجی

```
Python Programming
P
g
18
True
False
True
```

**نکات مهم**

- رشته‌ها در پایتون ( Immutableتغییرناپذیر (هستند؛ یعنی پس از ایجاد، نمی‌توان یک کاراکتر آن‌ها را مستقیماً تغییر داد
- اندیس‌گذاری رشته‌ها از صفر شروع می‌شود
- فاصله‌ها، اعداد و علائم نیز جزئی از رشته هستند و در محاسبه طول رشته توسط len()شمارش می‌شوند
- عملگر inوجود یک زیررشته را بررسی می‌کند و مقدار Trueیا Falseبرمی‌گرداند
- عملگر not inدقیقاً برعکس inعمل می‌کند

**۱۱-برش رشته‌ها (String Slicing) در زبان برنامه‌نویسی پایتون :**

یکی از قابلیت‌های بسیار مهم و پرکاربرد در زبان پایتون، برش رشته‌ها (String Slicing) است. با استفاده از Slicingمی‌توان بخشی از یک رشته را استخراج کرد، بدون اینکه رشته اصلی تغییر کند ، از آنجایی که رشته‌ها در پایتون مانند یک آرایه از کاراکترها (Strings are Arrays) هستند، می‌توان با استفاده از اندیس (Index) قسمت دلخواهی از آن‌ها را انتخاب کرد.

ساختار کلی برش رشته به صورت زیر است:

string[start:end]

در این ساختار:

- start : شماره اولین کاراکتر مورد نظر (شامل می‌شود)
- end : شماره آخرین کاراکتر (شامل نمی‌شود)

به عبارت دیگر، کاراکترهای از startتا قبل از endانتخاب می‌شوند.

**مثال**

```
text = "Python Programming"
print(text[0:6])
```

خروجی:

```
Python
```

در این مثال:

- شروع از اندیس ۰) حرف (P
- پایان در اندیس ۶ (حرفی با اندیس ۶ انتخاب نمی‌شود)

**۱۲-شماره اندیس‌های رشته :**

فرض کنید رشته زیر را داریم:

text = "Python"

| کاراکتر | P | y | t | h | o | n |
|---|---|---|---|---|---|---|
| اندیس مثبت | 0 | 1 | 2 | 3 | 4 | 5 |
| اندیس منفی | -6 | -5 | -4 | -3 | -2 | -1 |

**Slice From the Start-۱۳**

اگر مقدار startنوشته نشود، پایتون فرض می‌کند که برش از ابتدای رشته (اندیس صفر) آغاز می‌شود

ساختار:

string[:end]

**مثال**

```
b = "Hello, World!"
print(b[:5])
```

خروجی:

Hello

در این مثال:

b[:5] معادل است با b[0:5]

بنابراین کاراکترهای اندیس ۰ تا ۴ انتخاب می‌شوند

**مثال دیگر**

```
city = "Tehran"
print(city[:3])
```

خروجی:

Teh

### ۱۴-Slice To the End

اگر مقدار endنوشته نشود، پایتون فرض می‌کند که برش تا انتهای رشته ادامه دارد

ساختار:

string[start:]

**مثال**

```
b = "Hello, World!"
print(b[7:])
```

خروجی:

World!

**مثال دیگر**

```
language = "Python"
print(language[2:])
```

خروجی:

thon

زیرا از اندیس ۲ تا انتهای رشته انتخاب می‌شود

### ۱۵-حذف چند کاراکتر اول

نمونه کاربرد:

```
text = "Programming"
print(text[3:])
```

خروجی:

gramming

### ۱۶-برش بین دو اندیس

می‌توان ابتدا و انتهای برش را مشخص کرد

```
word = "Programming"

print(word[3:8])
```

خروجی:

```
gramm
```

### ۱۷-استفاده از اندیس‌های منفی(Negative Indexing)

اندیس‌های منفی شمارش را از انتهای رشته آغاز می‌کنند.

| اندیس | کاراکتر |
|---|---|
| -1 | آخرین کاراکتر |
| -2 | یکی مانده به آخر |
| -3 | سومین کاراکتر از انتها |

**مثال**

```
text = "Python"

print(text[-1])
```

خروجی:

```
n
```

مثال

```
print(text[-2])
```

خروجی:

```
o
```

**۱۸-برش با اندیس منفی**

در Slicing نیز می‌توان از اندیس منفی استفاده کرد

**مثال**

```
b = "Hello, World!"
print(b[-5:-2])
```

خروجی:

orl

توضیح:

Hello, World!

↑ ↑

-5 -2

کاراکترهای انتخاب‌شده:

o

r

l

حرف dانتخاب نمی‌شود، زیرا اندیس انتهایی در Slicing شامل نمی‌شود.

**مثال دیگر**

```
language = "Python"
print(language[-4:-1])
```

خروجی:

Tho

**۱۹-برش کل رشته**

```
text = "Python"
print(text[:])
```

خروجی:

```
Python
```

در این حالت، کل رشته کپی می‌شود

**۲۰-ترکیب اندیس مثبت و منفی**

استفاده از اندیس‌های مثبت و منفی با هم نیز امکان‌پذیر است

```
text = "Programming"
print(text[3:-2])
```

خروجی:

```
grammi
```

**۲۱-خطاهای رایج**

**اشتباه در انتخاب اندیس انتهایی**

```
text = "Python"
print(text[0:3])
```

خروجی:

```
Pyt
```

بعضی دانشجویان انتظار دارند:

```
Pyth
```

اما باید توجه داشت که اندیس انتهایی در Slicing انتخاب نمی‌شود

**استفاده از اندیس خارج از محدوده :**

```
text = "Python"
print(text[2:100])
```

خروجی:

```
thon
```

پایتون خطا نمی‌دهد و فقط تا انتهای رشته را برمی‌گرداند

**مثال جامع**

```
text = "Python Programming"
print(text[:6])
print(text[7:])
print(text[0:6])
print(text[7:18])
print(text[-11:-1])
print(text[:])
print(text[2:])
```

خروجی:

```
Python
Programming
Python
Programming
Programmin
Python Programming
thon Programming
```

**کاربردهایSlicing**

برش رشته‌ها در بسیاری از برنامه‌ها کاربرد دارد، از جمله:

- استخراج نام فایل
- استخراج پسوند فایل
- جدا کردن نام و نام خانوادگی
- استخراج سال، ماه و روز از تاریخ
- پردازش متن
- تحلیل داده‌های متنی
- پاک کردن چند کاراکتر اول یا آخر رشته

**نکات مهم**

- اندیس‌ها از صفر شروع می‌شوند
- درSlicing ، اندیس شروع (Start) انتخاب می‌شود، اما اندیس پایان (End) انتخاب نمی‌شود
- اگر startنوشته نشود، مقدار آن 0در نظر گرفته می‌شود
- اگر endنوشته نشود، برش تا انتهای رشته ادامه پیدا می‌کند
- اندیس‌های منفی از انتهای رشته شمارش می‌شوند
- اگر اندیس‌ها خارج از محدوده باشند، پایتون معمولاً خطا ایجاد نمی‌کند و نتیجه را تا حد امکان برمی‌گرداند

**۲۲-تغییر رشته‌ها (Modify Strings) در زبان برنامه‌نویسی پایتون :**

رشته‌ها (Strings) یکی از مهم‌ترین انواع داده در پایتون هستند. پس از ایجاد یک رشته، در بسیاری از برنامه‌ها نیاز داریم عملیات مختلفی مانند تبدیل حروف بزرگ و کوچک، حذف فاصله‌های اضافی، جستجو، جایگزینی، جدا کردن کلمات، بررسی محتوا و قالب‌بندی متن را روی آن انجام دهیم

پایتون مجموعه‌ای بسیار غنی از متدهای رشته (String Methods) را در اختیار برنامه‌نویس قرار می‌دهد که این کارها را به سادگی انجام می‌دهند . رشته‌ها در پایتون Immutable تغییرناپذیر هستند. بنابراین هیچ‌یک از متدهای رشته، متن اصلی را تغییر نمی‌دهند؛ بلکه یک رشته جدید ایجاد و برمی‌گردانند

**۲۳-تبدیل حروف به بزرگ(upper)**

متد ()upperتمام حروف رشته را به حروف بزرگ (Upper Case) تبدیل می‌کند

```
text = "Python Programming"
print(text.upper())
```

خروجی

```
PYTHON PROGRAMMING
```

**۲۴-تبدیل حروف به کوچک(lower)**

متد ()lowerتمام حروف رشته را به حروف کوچک (Lower Case) تبدیل می‌کند

```
text = "Python Programming"
print(text.lower())
```

خروجی

```
python programming
```

**۲۵-حذف فاصله‌های اضافی(Remove Whitespace)**

متد ()strip فاصله‌های ابتدا و انتهای رشته را حذف می‌کند

```
text = "    Python    "
print(text.strip())
```

خروجی

```
Python
```

توجه: فقط فاصله‌های ابتدا و انتهای رشته حذف می‌شوند و فاصله‌های بین کلمات باقی می‌مانند

### ۲۶-جایگزینی متن(replace)

متد ()replaceیک قسمت از رشته را با رشته دیگری جایگزین می‌کند

```
text = "I like Java"
print(text.replace("Java", "Python"))
```

خروجی

I like Python

### ۲۷-جدا کردن رشته(split)

متد ()split رشته را بر اساس جداکننده (Separator) به چند قسمت تقسیم کرده و یک **لیست (List)** برمی‌گرداند

```
text = "Python,Java,C++"
print(text.split(","))
```

خروجی

['Python', 'Java', 'C++']

### ۲۸-معرفی مهم‌ترین متدهای رشته

در ادامه، مهم‌ترین متدهای رشته در پایتون معرفی شده‌اند

| متد | توضیح |
|---|---|
| capitalize() | اولین حرف رشته را بزرگ و بقیه را کوچک می‌کند |
| casefold() | رشته را به حروف کوچک تبدیل می‌کند (قوی‌تر از ()lowerبرای مقایسه متون.( |
| center(width) | رشته را در یک عرض مشخص وسط‌چین می‌کند |
| count(value) | تعداد دفعات تکرار یک عبارت را برمی‌گرداند |
| encode() | رشته را به داده‌های باینری (Bytes) تبدیل می‌کند |
| endswith() | بررسی می‌کند که آیا رشته با عبارت مشخصی پایان می‌یابد یا خیر |

| | |
|---|---|
| expandtabs() | اندازه Tab را به تعداد مشخصی فاصله تبدیل می‌کند |
| find() | محل اولین وقوع یک عبارت را پیدا می‌کند؛ در صورت نیافتن مقدار 1-برمی‌گرداند. |
| format() | مقادیر را درون رشته قالب‌بندی می‌کند |
| format_map() | مشابه format()، اما با استفاده از دیکشنری |
| index() | محل اولین وقوع عبارت را پیدا می‌کند؛ اگر پیدا نشود، خطا ایجاد می‌شود |
| isalnum() | بررسی می‌کند که همه کاراکترها حرف یا عدد باشند |
| isalpha() | بررسی می‌کند که همه کاراکترها فقط حروف باشند |
| isascii() | بررسی می‌کند که همه کاراکترها در مجموعه ASCII باشند |
| isdecimal() | بررسی می‌کند که رشته فقط شامل ارقام ده‌دهی باشد |
| isdigit() | بررسی می‌کند که رشته فقط شامل ارقام باشد |
| isidentifier() | بررسی می‌کند که رشته یک نام معتبر برای متغیر باشد |
| islower() | بررسی می‌کند که تمام حروف کوچک باشند |
| isnumeric() | بررسی می‌کند که رشته فقط شامل مقادیر عددی باشد |
| isprintable() | بررسی می‌کند که تمام کاراکترها قابل چاپ باشند |
| isspace() | بررسی می‌کند که رشته فقط از فاصله‌ها تشکیل شده باشد |
| istitle() | بررسی می‌کند که اولین حرف هر کلمه بزرگ باشد |
| isupper() | بررسی می‌کند که تمام حروف بزرگ باشند |
| join() | عناصر یک لیست یا مجموعه را به یک رشته تبدیل می‌کند |
| ljust(width) | رشته را در عرض مشخص، از سمت چپ تراز می‌کند |
| lower() | تمام حروف را کوچک می‌کند |
| lstrip() | فاصله‌های ابتدای رشته را حذف می‌کند |
| maketrans() | جدول تبدیل برای متد translate() ایجاد می‌کند |

| | |
|---|---|
| partition() | رشته را به سه قسمت تقسیم می‌کند: قبل از جداکننده، خود جداکننده و بعد از آن |
| replace() | جایگزین کردن بخشی از رشته با متن جدید |
| rfind() | محل آخرین وقوع یک عبارت را پیدا می‌کند |
| rindex() | مشابه rfind()، اما در صورت نیافتن، خطا ایجاد می‌کند |
| rjust(width) | رشته را از سمت راست تراز می‌کند |
| rpartition() | مشابه partition()اما از انتهای رشته جستجو می‌کند |
| rsplit() | رشته را از سمت راست تقسیم می‌کند |
| rstrip() | فاصله‌های انتهای رشته را حذف می‌کند |
| split() | رشته را به لیستی از زیررشته‌ها تقسیم می‌کند |
| splitlines() | رشته را بر اساس خطوط جدید (\n) تقسیم می‌کند |
| startswith() | بررسی می‌کند که رشته با عبارت مشخصی شروع شود |
| strip() | فاصله‌های ابتدا و انتهای رشته را حذف می‌کند |
| swapcase() | حروف بزرگ را کوچک و حروف کوچک را بزرگ می‌کند |
| title() | اولین حرف هر کلمه را بزرگ می‌کند |
| translate() | با استفاده از جدول maketrans()کاراکترها را تبدیل می‌کند |
| upper() | تمام حروف را بزرگ می‌کند |
| zfill(width) | در ابتدای رشته صفر اضافه می‌کند تا طول آن به مقدار مشخص برسد |

**نمونه‌هایی از متدهای پرکاربرد**

**capitalize()** :

```
text = "python programming"
print(text.capitalize())
```

خروجی

```
Python programming
```

**count()** :

```
text = "banana"
print(text.count("a"))
```

خروجی

```
3
```

**startswith()** :

```
text = "Python Programming"
print(text.startswith("Python"))
```

خروجی

```
True
```

**endswith()** :

```
text = "report.pdf"
print(text.endswith(".pdf"))
```

خروجی

```
True
```

: **find()**

```
text = "Python Programming"
print(text.find("Program"))
```

خروجی

7

: **join()**

```
languages = ["Python", "Java", "C++"]
print(" - ".join(languages))
```

خروجی

Python - Java - C++

: **title()**

```
text = "python programming language"
print(text.title())
```

خروجی

Python Programming Language

: **swapcase()**

```
text = "Python"
print(text.swapcase())
```

خروجی

pYTHON

**zfill()** :

```
number = "25"

print(number.zfill(5))
```

خروجی

```
00025
```

**متدهای پرکاربرد در برنامه‌نویسی روزمره**

اگرچه پایتون بیش از ۴۰ متد برای رشته‌ها دارد، اما در عمل برنامه‌نویسان بیشتر از متدهای زیر استفاده می‌کنند:

| متد | کاربرد |
|---|---|
| upper() | تبدیل به حروف بزرگ |
| lower() | تبدیل به حروف کوچک |
| strip() | حذف فاصله‌های اضافی |
| replace() | جایگزینی متن |
| split() | تبدیل رشته به لیست |
| join() | تبدیل لیست به رشته |
| find() | جستجوی یک عبارت |
| count() | شمارش تعداد تکرار |
| startswith() | بررسی شروع رشته |
| endswith() | بررسی پایان رشته |
| title() | بزرگ کردن اولین حرف هر کلمه |
| capitalize() | بزرگ کردن اولین حرف رشته |

| | |
|---|---|
| isalpha() | بررسی حروف بودن رشته |
| isdigit() | بررسی عدد بودن رشته |
| isalnum() | بررسی حرف یا عدد بودن رشته |

**الحاق رشته‌ها(String Concatenation)**

در بسیاری از برنامه‌ها لازم است متن‌های مختلف را به یکدیگر متصل کنیم، مقادیر متغیرها را داخل متن قرار دهیم یا کاراکترهای خاصی مانند خط جدید، تب یا علامت نقل‌قول را نمایش دهیم. پایتون امکانات بسیار قدرتمندی برای انجام این کارها در اختیار برنامه‌نویسان قرار می‌دهد

در این بخش با مفاهیم زیر آشنا می‌شویم:

- String Concatenation
- Format Strings
- F-Strings
- Placeholders
- Escape Characters

**الحاق رشته‌ها(String Concatenation)**

**Concatenation**به معنی اتصال دو یا چند رشته به یکدیگر است در پایتون این کار با عملگر  +انجام می‌شود.

**مثال**

```
first_name = "Ali"
last_name = "Ahmadi"
full_name = first_name + last_name
print(full_name)
```

خروجی

AliAhmadi

**اضافه کردن فاصله :**

```
first_name = "Ali"
last_name = "Ahmadi"
full_name = first_name + " " + last_name
print(full_name)
```

خروجی

Ali Ahmadi

**اتصال چند رشته :**

```
language = "Python"
version = "3.13"
text = language + " " + version
print(text)
```

خروجی

Python 3.13

**خطای رایج در Concatenation :** اتصال مستقیم رشته و عدد مجاز نیست

```
age = 20
print("Age = " + age)
```

خروجی

TypeError

برای رفع این مشکل باید از str() استفاده کنیم.

```
age = 20
print("Age = " + str(age))
```

**قالب‌بندی رشته‌ها(Format Strings)** :

در بسیاری از برنامه‌ها لازم است مقدار متغیرها را داخل یک متن قرار دهیم ، یکی از روش‌های قدیمی استفاده از متد format() است

**مثال**

```
age = 20
text = "I am {} years old."
print(text.format(age))
```

خروجی

```
I am 20 years old.
```

**چندPlaceholder** :

```
name = "Ali"
age = 20
text = "My name is {} and I am {} years old."
print(text.format(name, age))
```

خروجی

```
My name is Ali and I am 20 years old.
```

**استفاده از شمارهPlaceholder** :

```
name = "Ali"
age = 20
text = "Age: {1} Name: {0}"
print(text.format(name, age))
```

خروجی

```
Age: 20 Name: Ali
```

**F-Strings** :

از نسخه Python 3.6به بعد، روش جدیدی به نام Formatted String Literalsیا F-Stringsمعرفی شد این روش سریع‌تر، ساده‌تر و خواناتر از format()است ، برای استفاده، قبل از رشته حرف fقرار می‌دهیم.

**مثال**

```
name = "Ali"
print(f"Hello {name}")
```

خروجی

```
Hello Ali
```

**استفاده از چند متغیر :**

```
name = "Ali"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

**قرار دادن عبارت ریاضی :**

```
a = 10
b = 20
print(f"Sum = {a+b}")
```

خروجی

```
Sum = 30
```

**Placeholderچیست؟**

قسمتی از رشته که مقدار یک متغیر یا عبارت در آن قرار می‌گیرد، Placeholderنام دارد ، در F-Strings هر چیزی که داخل {}قرار گیرد، محاسبه شده و نتیجه آن نمایش داده می‌شود.

**Placeholderمی‌تواند شامل موارد زیر باشد**

**متغیرها**

```
name = "Python"
print(f"Language: {name}")
```

**عملیات ریاضی**

```
x = 15
y = 5
print(f"{x} + {y} = {x+y}")
```

**توابع**

```
name = "python"
print(f"{name.upper()}")
```

خروجی

PYTHON

**محاسبات پیچیده**

```
length = 10
width = 8
print(f"Area = {length * width}")
```

**استفاده از Modifier**

Modifierنحوه نمایش مقدار را تغییر می‌دهد

```
price = 12.56789

print(f"{price:.2f}")
```

خروجی

12.57

در اینجا

:.2f

یعنی فقط دو رقم اعشار نمایش داده شود

**نمونه‌های دیگر Modifier**

```
number = 1250

print(f"{number:,}")
```

خروجی

1,250

```
number = 25

print(f"{number:05}")
```

خروجی

00025

**Escape Characters** :

گاهی لازم است کاراکترهایی را نمایش دهیم که معنی خاصی برای پایتون دارند در این حالت از Escape Characterاستفاده می‌کنیم همه Escape Character ها با علامت \ شروع می‌شوند.

**مهم‌ترین Escape Character ها**

| کد | معنی | مثال |
|---|---|---|
| \' | نمایش کوتیشن تکی | I\'m |
| \" | نمایش کوتیشن دوتایی | \"Python\" |
| \\ | نمایشBackslash | C:\\Users\\Ali |
| \n | رفتن به خط جدید | چاپ چندخطی |
| \r | بازگشت به ابتدای خط(Carriage Return) | کاربردهای خاص |
| \t | ایجادTab | ایجاد فاصله منظم |
| \b | حذف یک کاراکتر قبل(Backspace) | کاربرد محدود |
| \f | Form Feed | چاپگرها و فایل‌های متنی |
| \ooo | نمایش کاراکتر با مقدارOctal | \101 → A |
| \xhh | نمایش کاراکتر با مقدارHex | \x41 → A |

**استفاده از نقل قول**

```
print("I'm learning Python")
```

یا

```
print('I\'m learning Python')
```

**نمایش Backslash**

```
print("C:\\Users\\Ali")
```

خروجی

```
C:\Users\Ali
```

**خط جدید**

```
print("Python\nJava\nC++")
```

خروجی

```
Python
Java
C++
```

**Tab**

```
print("Name\tAge\tCity")
```

خروجی

```
Name    Age    City
```

**Backspace**

```
print("ABC\bD")
```

خروجی تقریباً

```
ABD
```

**Octal**

```
print("\101")
```

خروجی

```
A
```

**Hexadecimal**

```
print("\x41")
```

خروجی

```
A
```

**مثال جامع**

```
name = "Ali"
age = 21
city = "Tehran"
print(f"My name is {name}.")
print(f"I am {age} years old.")
print(f"I live in {city}.")
print(f"Next year I will be {age+1} years old.")
print("Languages:\nPython\nJava\nC++")
print("Name\tAge\tCity")
print("C:\\Users\\Ali")
```

**مقایسه روش‌های قالب‌بندی رشته**

| روش | مثال | وضعیت |
|---|---|---|
| Concatenation | "Hello " + name | ساده، اما برای متن‌های طولانی مناسب نیست. |
| format() | "Hello {}".format(name) | خوانا و سازگار با نسخه‌های قدیمی پایتون |
| **F-Strings** | f"Hello {name}" | **بهترین و توصیه‌شده برای Python 3.6 به بعد** |

**نکات مهم**

- برای اتصال رشته‌ها از عملگر **+**استفاده می‌شود
- اتصال مستقیم رشته و عدد مجاز نیست؛ ابتدا باید عدد را با str()به رشته تبدیل کرد یا از F-Strings استفاده نمود
- در F-Stringsهر عبارت داخل {}محاسبه شده و نتیجه آن نمایش داده می‌شود
- Placeholderمی‌تواند شامل متغیرها، عملیات ریاضی، توابع وModifier ها باشد

- Escape Charactersبا علامت \شروع می‌شوند و برای نمایش کاراکترهای خاص مانند خط جدید (\n) و تب (\t) استفاده می‌شوند